



Original article

WAScope: Detecting privacy data leakage with web application-specific API confusion

Yu Nie ^{a,b}, Jianming Fu ^a, Xinghang Lv ^a, Chao Li ^b, Shixiong Yang ^b, Guojun Peng ^a

^a Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan, China

^b School of Information Engineering, Jingdezhen Ceramic University, Jingdezhen, China

ARTICLE INFO

Keywords:

Web application-specific API
Privacy leakage
API confusion
Defense

ABSTRACT

The number of web applications deployed on the internet has exceeded one billion, accumulating vast amounts of user privacy data. The compromise of such data may lead to severe consequences. While existing research has primarily focused on data exfiltration through system APIs, the security risks posed by application-specific APIs have been largely overlooked. These APIs directly manage the collection, processing, and transmission of sensitive user data, making them critical attack surfaces. This study systematically investigates privacy data leakage caused by unauthorized access through web application-specific APIs. We presented WAScope (*Web Application-specific API Scope*), a dynamic analysis tool that detects privacy leakage by combining API confusion techniques with a customized privacy dictionary. We conducted experiments on 100 real-world web applications using WAScope. The tool identified 15,593 privacy-aware API data flows across 76 applications, among which 2,757 APIs were manually confirmed to expose sensitive data due to improper access controls. Manual validation further validated the findings, revealing a 9% false positive rate. We reported these vulnerabilities to the China National Vulnerability Database (CNVD), receiving 10 official CNVD-IDs that demonstrate the effectiveness of WAScope.

1. Introduction

According to Forbes [1], over 1 billion web applications are hosted on the internet, serving users worldwide. These applications routinely collect and store substantial volumes of privacy-sensitive user data during operation, including personally identifiable information (PII), financial records, and social communication data. The centralized storage of such sensitive information significantly increases the risk of data breaches, which may result in severe privacy violations and financial losses. A 2022 report by Trend Micro [2] documented personal information leakage cases caused by vulnerabilities in the web APIs of e-commerce applications. Recent developments in 2023 saw frequent data breach incidents within China: regulatory sanctions were imposed on affected entities, including a higher education institution in Jiangxi Province [3], a municipal e-governance service provider in Shanghai [4], and a real estate registry center in Taizhou City [5], all stemming from inadequate access control mechanisms. These escalating risks underscore the urgent need for developers and operators to strengthen access controls and address privacy leakage risks in web applications.

In the OWASP [6] Top Ten Web Application Risks report, 94% of tested web applications were found to exhibit vulnerabilities in access control policies, particularly the unauthorized exposure of sensitive data. With the widespread adoption of modern architectures like single-page applications (SPA) and microservices [7], API utilization in web applications has grown exponentially. APIs in web applications can be categorized into two distinct types: system APIs and application-specific APIs. System APIs are standardized by the technical specifications of the W3C [8] and WHATWG [9], with attackers primarily exploiting client-side APIs (such as XMLHttpRequest [10] or the Fetch API [11]) to conduct attacks including arbitrary code execution, cross-site request forgery (CSRF [12]), and open redirects. In contrast, application-specific APIs are custom-developed for direct interaction with user data. These APIs serve as core components for managing user privacy data, thereby becoming high-priority targets for data theft attempts.

However, existing detection tools and vulnerability scanners such as OWASP ZAP [13], Burp Suite [14], VOAPI² [15], and Sheriff [16] either rely on static rule-based matching or require significant manual analysis. These methods struggle with non-standard API designs

* Corresponding author.

E-mail addresses: yu.nie@whu.edu.cn (Y. Nie), jmfu@whu.edu.cn (J. Fu), lvxinghang@whu.edu.cn (X. Lv), lichao@jcu.edu.cn (C. Li), yangshixiong@stu.jcu.edu.cn (S. Yang), guojpeng@whu.edu.cn (G. Peng).

<https://doi.org/10.1016/j.aej.2025.08.006>

Received 6 June 2025; Received in revised form 19 July 2025; Accepted 4 August 2025

Available online 26 August 2025

1110-0168/© 2025 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Alexandria University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

and application-specific data fields, leading to low detection rates. Furthermore, prior work has largely overlooked semantic diversity in data representation, protocol heterogeneity, and the need for automated, large-scale detection across varied web application types. The widespread use of diverse architectures and custom data formats in modern web applications introduces greater variability in API semantics and structures, complicating the detection of privacy data leakage. Consequently, many privacy vulnerabilities remain undetected by existing tools, which are typically designed for structured RESTful interfaces or rely on known keyword patterns.

Successful exploitation requires attackers to craft malicious requests through parameter or path manipulation during authenticated sessions. Prior research has predominantly examined system APIs—for example, asynchronous API calls initiated via XMLHttpRequest or the Fetch API—with recent studies extending coverage to emerging protocols, including push notifications [17], WebSockets [18], and server-sent event channels such as the sendBeacon API [19]. Furthermore, scholarly attention has primarily focused on unauthorized access patterns within RESTful-compliant interfaces when analyzing application-specific APIs, while security analysis of non-RESTful implementations remains critically underexplored. This gap poses a significant challenge, as many web applications use non-standard API designs, which may introduce unique vulnerabilities not detectable by conventional methods.

In this paper, we systematically investigate application-specific APIs in web applications, aiming to identify and quantify privacy data leakage issues caused by improperly enforced access control policies. The research primarily consists of two aspects: First, we analyze the causes of privacy data leakage in application-specific APIs and find that such leakage primarily occurs during interactions between these APIs and users. Violations of access control policies during these interactions are identified as the direct cause of leakage. Second, we implement an automated method for identifying privacy data leaks in application-specific APIs. Our findings show that we resolve the semantic ambiguity of privacy data by constructing a localized synonym-based privacy dictionary. For detecting unauthorized API access, we implemented automated detection of access risks in application-specific APIs using the *API Confusion* method. Ultimately, by combining data differential analysis with the privacy dictionary's matching mechanism, we successfully developed an automated privacy leakage detection tool, effectively addressing the challenge of detecting privacy data leakage in web applications. Given the persistence of access control misconfigurations across development cycles, our work also lays the foundation for future research on longitudinal auditing and proactive security maintenance.

Contributions. To summarize, this paper makes the following contributions:

- **Privacy-aware APIs Identification:** We propose a systematic approach for identifying privacy data requests in web application-specific APIs. By leveraging a custom-built privacy data dictionary, our method accurately identifies APIs that process privacy-related data.
- **Privacy Leakage Detection:** We propose a novel detection framework for uncovering privacy data leakage resulting from unauthorized access. This approach transforms original application-specific APIs into confused counterparts using the API Confusion technique, and then conducts differential analysis between the responses of the original and confused APIs. The resulting differences, interpreted through the privacy dictionary, reveal potential privacy leakages.
- **Empirical Validation:** We have developed *WAScope* as an automated detection tool. We evaluated 100 real-world web applications and identified 2757 APIs with potential privacy risks. Through responsible disclosure to CNVD and direct communication with the respective web administrators, we obtained 10 CNVD IDs verifying confirmed 0 day vulnerabilities.

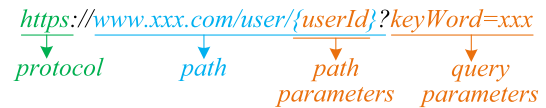


Fig. 1. Web application-specific API structure.

2. Background and motivation

Structural Overview: We examine the structure of web application-specific APIs (Section 2.1) and their associated unauthorized access vulnerabilities (Section 2.2). Subsequently, the penetration testing case study (Section 2.3) demonstrates how such unauthorized access enables privacy leakage through these APIs. We conclude with a discussion of the capabilities (Section 2.4) and challenges (Section 2.5) involved in conducting such security assessments.

2.1. Web application-specific API structure

Developers define web application-specific APIs as consisting of a protocol, path, request method, parameters, authentication/authorization credentials, and response, as shown in Fig. 1.

- **Protocol:** Common protocols for web application-specific APIs include *HTTP/HTTPS* and *WebSocket*.
- **Path:** API paths adhere to URI specifications, include *base path* (e.g., “https://xxx.com/v1”) and a *resource path* (e.g., “/users/{userId}/details”), where “{userId}” denotes a path parameter.
- **Request Method:** Request methods used by web application-specific APIs include *GET*, *POST*, *PUT*, and *DELETE*.
- **Parameters:** Parameters of web application-specific APIs include *path parameters* (e.g., “{userId}” in “/users/{userId}/details”), *query parameters* (e.g., “page” in “?page=2”), *header parameters* (e.g., “Authorization credentials”), and *request body parameters* (typically used in POST and PUT methods).
- **Authentication/Authorization Credential:** Common credentials for web application-specific APIs include *API Key*, *OAuth 2.0*, *JWT* (JSON Web Token), and *Basic Authentication*; these credentials are typically transmitted via header parameters.
- **Response:** Responses include the *HTTP status code* (e.g., 200), *response header* (e.g., Content-Type), and *response body* (e.g., JSON).

2.2. Unauthorized access vulnerabilities

Unauthorized access vulnerabilities (UAVs) [20] pose severe risks to API-driven privacy protection in modern web applications. These vulnerabilities enable information leakage, permitting attackers to overstep their assigned permissions. Exploitation typically progresses through three phases: (1) vulnerability discovery, (2) privilege escalation, and (3) sensitive data exfiltration. Empirical evidence shows that such attacks can lead to large-scale privacy violations, compromising PII, financial records, and critical infrastructure [21,22].

In the security architecture of web system APIs, attackers typically exploit vulnerabilities by intercepting insufficiently validated requests and bypassing the Same-Origin Policy (SOP) constraints [16], thereby gaining unauthorized access to sensitive data.

In contrast, unauthorized access vulnerabilities in web application-specific APIs stem from failures in access control mechanisms at the business logic layer. Once the attacker acquires a valid application session token, they can craft a malicious request to circumvent the web application's access control policy and illegitimately retrieve sensitive data.

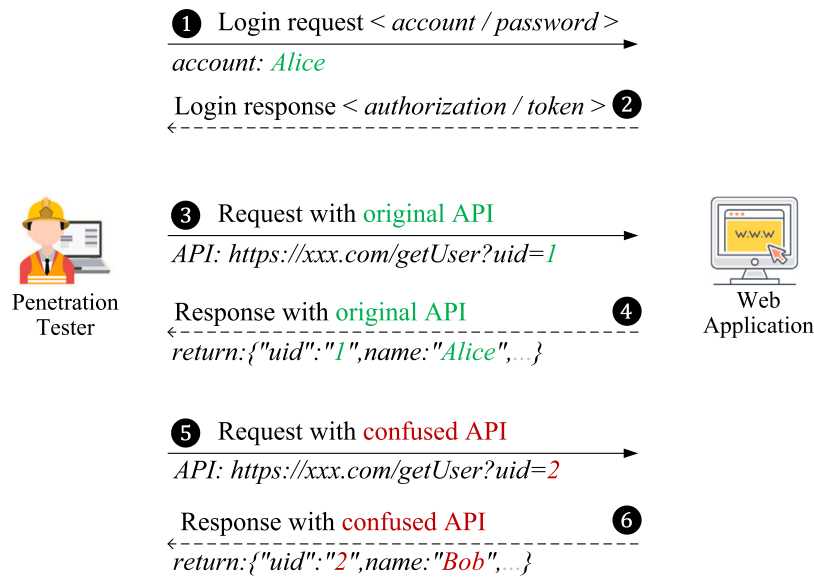


Fig. 2. Penetration test case of a privacy data leakage caused by a web application-specific API.

2.3. Penetration test case

The penetration tester initially logs in to the web application using Alice's account credentials (Step ① in Fig. 2). Upon successful authentication, the system returns user authorization credentials in the response (Step ② in Fig. 2). The session state established during this phase serves as the basis for authenticating subsequent requests.

During testing, the penetration tester discovers API requests used to retrieve user privacy data (Step ③ in Fig. 2). The web application subsequently returns a response containing Alice's privacy data (Step ④ in Fig. 2).

Notably, when the tester performs API confusion against the privacy-aware API—such as tampering with the `uid` parameter—the server's flawed authorization logic may return responses exceeding the current user's entitlements. By modifying the `uid` parameter in crafted requests (Step ⑤ in Fig. 2), the tester successfully exfiltrates Bob's sensitive information (Step ⑥ in Fig. 2), which lies outside Alice's authorized access scope.

This case study demonstrates the risk of private data leakage resulting from unauthorized access to web application-specific APIs.

2.4. Capability

Through the analysis of privacy leakage processes in the penetration test case (Section 2.3), we delineate the penetration tester's capabilities as follows:

- **HTTP Data Interception.** Upon accessing a web application, the tester can intercept authorization-related HTTP traffic, capturing APIs, request payloads, and responses through protocol analysis.
- **Privacy-aware API Identification.** The tester can identify web application-specific APIs involving privacy data and assess their susceptibility to API confusion.
- **Privilege Escalation.** After identifying privacy-aware APIs, the tester systematically manipulates API parameters and paths to replay requests for privileged data acquisition. Compromised data enables downstream threats, including user profiling [23,24], network reconnaissance [25,26], and social engineering campaigns [27–29].

2.5. Challenge

- **Challenge 1: Identification of Privacy-aware APIs.** Within web application-specific API requests, not all APIs involve privacy data. Therefore, determining whether privacy data is embedded within numerous web application-specific API requests poses a significant challenge.
- **Challenge 2: API Confusion.** In order to achieve automated penetration testing, the paths and parameters of specific APIs need to be automatically and thoroughly manipulated (such as path tampering and parameter mutation), especially while maintaining functional effectiveness during unauthorized access attempts; this presents a considerable challenge.
- **Challenge 3: Risk Detection.** The presence of privacy data in web application-specific APIs does not inherently imply leakage risks. Therefore, detecting potential privacy data leakage vulnerabilities within these APIs by analyzing responses from numerous confused requests poses significant technical challenges.

3. Method

Penetration Test. Following cybersecurity ethical guidelines and compliance frameworks, penetration testers obtain explicit authorization from the web application operator, which defines the scope of penetration testing, permissible attack vectors, and data protection clauses. This authorization legally binds testing activities to the operator's authorized systems and designated timeframes to mitigate legal liabilities while establishing a legal foundation for technical operations. Upon finalizing the binding terms of the agreement, testers must authenticate using pre-authorized valid credentials (e.g., account/password authentication) to establish a controlled testing environment with baseline permissions. The testers then execute penetration testing, proactively assessing risks, including data transmission flows, authentication mechanisms, and access control policies. Upon identifying unauthorized data access (e.g., horizontal privilege escalation), sensitive data leakage, or logical flaws, testers must promptly submit detailed technical reports with complete evidentiary chains to the operator. During remediation, testers collaborate on validating repair plans' feasibility and perform re-testing post-patch implementation,

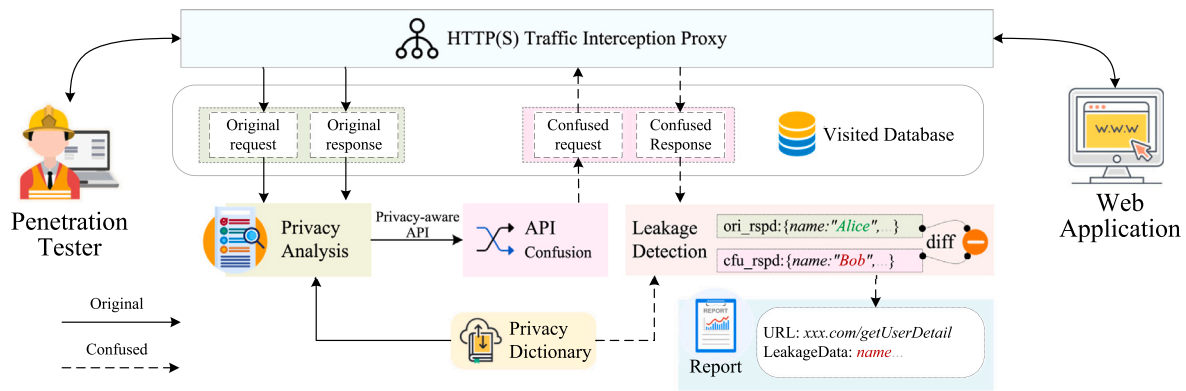


Fig. 3. WAScope overview.

thereby establishing a closed-loop security verification process from “vulnerability discovery to fix validation”.

In penetration testing targeting privacy data leakage issues in web applications, testers face the three technical challenges described in Section 2.5. This study proposes solutions addressing these challenges as follows:

To address the first challenge, traffic capturing on web application-specific APIs is performed to comprehensively collect request data (parameters and paths) and response data, constructing a normalized storage-based access log database; privacy-aware API identification is achieved by parsing request parameters, path schemas, and response payloads.

Regarding the second challenge, obfuscation strategies targeting core data carriers (parameters and paths) are implemented: input mutation techniques—such as enumeration, incremental modification, and random deletion—are applied to ensure confused API requests maintain functional validity while triggering latent unauthorized access attempts.

For the third challenge, an efficient differential response analysis-based risk detection algorithm is designed: the target API’s potential privacy leakage risks are validated by analyzing the differential between the original response and the confused response.

We developed a tool called WAScope, comprising four components: Traffic Interception Proxy, Privacy Analysis, API Confusion, and Leakage Detection, as shown in Fig. 3.

3.1. Privacy dictionary

We rely extensively on privacy dictionaries in privacy analysis and risk detection tasks. Our dictionary construction method is grounded in the Data Security Law of the People’s Republic of China (DSLPRC) [30] and the General Data Protection Regulation (GDPR) [31]. We initially defined four core categories of privacy data: Personally Identifiable Information (PII), Social Communication, Finance, and Documents.

Empirical observations reveal that web applications in the government and education sectors store massive volumes of user privacy data. Such data is highly susceptible to cross-platform correlation analysis, enabling the construction of precise user profiles. Given the criticality of these services in safeguarding public interests and individual rights, we selected government and education web applications as primary research targets.

Through manual analysis of government and education service web application in China, our privacy dictionary was populated with 70 privacy data items, including 44 PII items, 9 Social Communication Data items, 13 Financial Transaction Data items, and 4 Document-based Data items. We integrated GDPR special categories (e.g., iris scans, palm prints) into PII while excluding criminal records. We removed

DSLPRC-prohibited data posing national security risks (e.g., military facility geolocation data).

A critical finding emerged: localized semantic conventions dominate field naming in Chinese government and education systems. For instance, “sfz” (from the Chinese Pinyin “shenfenzheng”) represents “ID card”. To address semantic diversity challenges, we retrieved 100 open-source projects from Gitee [32] (China’s open-source platform) in government and education domains; expanded the dictionary to 210 privacy data items via NLP (Natural Language Processing) techniques and manual auditing; and implemented localized synonym extensions for Chinese Pinyin-based abbreviations, as shown in Table 1.

To ensure the privacy dictionary is applicable to web applications across different countries and regions, we designed it as a key–value pair structure in JSON format. For example, the data entry for identity cards is defined as: `{“IdentityCard”: [“idcard”, “shenfenzheng”, “sfz”, ...]}`. To support localized semantics in other countries or regions, it is only necessary to expand the corresponding entries in the dictionary, without modifying the detection logic.

3.2. Traffic interception proxy

This component is primarily responsible for intercepting and capturing HTTP(S) request traffic. The collected data contains authentication information (such as *tokens* and *cookies*), request headers (such as *referer*, *hostname*, and *port*), specific API metadata (such as *path*, *request method*, and *URL*), and their corresponding response data. The initially captured request/response data undergoes semantic analysis based on a privacy dictionary. Identified privacy-aware APIs then undergo obfuscation operations, with the confused response data serving as the basis for privacy leakage detection.

3.3. Privacy analysis

Analysis of data acquired through the HTTP traffic interception component reveals that when the path or parameters of a specific API contain privacy data fields, the corresponding response data typically includes privacy data. Notably, although many business-type APIs do not contain explicit privacy fields, their response data may still include user privacy. Based on this observation, our fundamental data source for privacy consists of the specific APIs’ paths, parameters, and responses. The privacy-aware API sequences generated through subsequent analysis will be used for obfuscation operations.

3.4. API confusion

API Confusion refers to the structured generation of mutated API requests by applying path and parameter obfuscation techniques to the original API sequences identified during privacy analysis. This process

Table 1
Partial contents of the privacy dictionary.

Category	Items	English	Chinese Pinyin
Personal identity information	Real name	<i>name</i>	<i>xingming</i>
	Identity card	<i>idcard</i>	<i>shenfenzheng</i>
	Birthday	<i>birth</i>	<i>shengri</i>
Social comm.	Email	<i>mail</i>	<i>youxiang</i>
	Telephone	<i>tel</i>	<i>dianhua</i>
	Mobile	<i>phone</i>	<i>shouji</i>
Finance	Bank accounts	<i>bank</i>	<i>yinhang</i>
	Insurance	<i>insurance</i>	<i>baoxian</i>
Documents	Auth	<i>file</i>	<i>wenjian</i>
	Cert	<i>cert</i>	<i>zhengshu</i>
	Other	<i>attach</i>	<i>fujian</i>

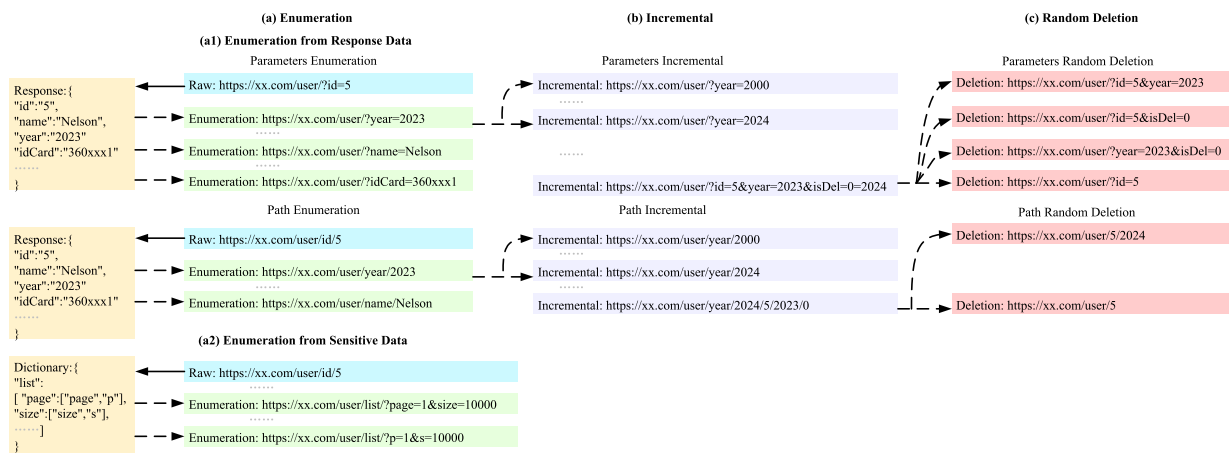


Fig. 4. Web application-specific API obfuscation operations.

produces a comprehensive set of candidate requests that simulate unauthorized behaviors, serving as inputs for subsequent leakage detection and significantly enhancing test coverage without requiring access to source code. These confusion operations follow a three-stage processing flow (Algorithm 1 provides a detailed description of the overall API obfuscation procedure):

- 1. Enumeration.** Firstly, we systematically enumerate and replace the paths and parameters of the target API. As demonstrated in the case illustrated in Fig. 4(a1), we achieve dynamic substitution of API elements by extracting variable-value pairs from the privacy response data. This processing can modify the construction of SQL query conditions and effectively expand the coverage of test logic. Additionally, as depicted in Fig. 4(a2), we establish semantic mapping relationships such as “list” → “{size, page}” and utilize the path keyword library collected from open-source platforms to replace sensitive words, thereby enhancing the coverage of API testing.
- 2. Incremental Phase.** As shown in Fig. 4(b), boundary value incremental processing is implemented for the numerical characteristics of API elements. This strategy can effectively uncover IDOR (Insecure Direct Object Reference) vulnerabilities and detect the risk of unauthorized data access.
- 3. Random Deletion Phase.** After generating the candidate list, a mirror set is created through a copy operation, and structured random deletion is performed on the paths and parameters. As presented in the case shown in Fig. 4(c), this processing can

be equivalently regarded as weakening SQL query condition restrictions to achieve maximum coverage of the query range.

By systematically transforming the request space through enumeration, incremental mutation, and random parameter deletion, the algorithm synthesizes a wide range of test cases that simulate unauthorized access attempts. This strategy improves detection coverage while minimizing reliance on source code or manual inspection, and providing a structured input foundation for subsequent leakage detection.

3.5. Leakage detection

After replaying the confused API request, WAScope performs leakage detection analysis based on the privacy dictionary. Firstly, it compares the hash values of the original response and each corresponding confused response. If they are identical, the response is marked as safe. When a hash difference is detected, WAScope automatically performs privacy data analysis on the confused response and determines whether it contains privacy data by matching it against entries in the privacy dictionary. This detection mechanism can accurately identify the data leakage path of the web application API and trace it back to the source request page, forming a complete risk evidence chain. The leakage detection algorithm is shown in Algorithm 2.

During the leakage detection process, penetration testers log into the target web application via WAScope and use a Breadth-First Search (BFS) algorithm to traverse the page's hierarchical navigation. To balance test coverage with execution efficiency, the traversal depth is set to three layers. During traversal, each triggered functional page generates multiple original API requests. Notably, due to functional coupling

Algorithm 1 API Confusion

Input: *OriginAPIs*: List of Privacy-aware APIs.
CandCtrls: Candidate control strategies (EnumReplaceSet, IncRange[m,n], RandDCount).
Output: *ConfusedAPIs*: Candidate API collection after obfuscation operations.

```

1: ConfusedAPIs  $\leftarrow \emptyset$ 
2: for each API  $\in$  OriginAPIs do
3:   // Step1: Enumeration
4:   for each (Path, Params)  $\in$  CandCtrls.EnumReplaceSet do
5:     newPath  $\leftarrow$  Path.replace(API.Path)
6:     newPara  $\leftarrow$  Para.replaceValue(API.Para)
7:     TmpSet1  $\leftarrow$  CartesianProduct(newPath, newPara)
8:   end for
9:   // Step2: Incremental
10:  for each API  $\in$  TmpSet1 do
11:    if isNumeric(API.Path) then
12:      TmpSet2  $\leftarrow \{API.Path.value \pm k \mid k \in [m, n]\}$ 
13:    end if
14:    if isNumeric(API.Para) then
15:      TmpSet2  $\leftarrow \{API.Para.value \pm k \mid k \in [m, n]\}$ 
16:    end if
17:  end for
18:  TmpSet0  $\leftarrow$  TmpSet1  $\cup$  TmpSet2
19:  // Step3: Random deletion
20:  TmpSet3  $\leftarrow$  Copy(TmpSet0)
21:  for each Cand  $\in$  TmpSet3 do
22:    if Rand(0, 1) < CandCtrls.RandDCount then
23:      DelParams  $\leftarrow$  RandSubset(Cand.Params)
24:      DelPaths  $\leftarrow$  RandSegment(Cand.Path)
25:      Cand.Remove(DelParams, DelPaths)
26:    end if
27:  end for
28:  ConfusedAPIs  $\leftarrow$  TmpSet0  $\cup$  TmpSet3
29: end for
30: return ConfusedAPIs

```

Algorithm 2 Privacy Data Leakage Risk Detection

Input: *ORd*: Origin API Response data,
CRd: Confusion APIs Response data,
Pd: Privacy dictionary
Output: *Boolean*;

```

1: Result  $\leftarrow$  False
2: for each c  $\in$  CRd do
3:   if hash(c)  $\neq$  hash(ORd) then
4:     for each field  $\in$  c do
5:       if field  $\in$  Pd then
6:         Result  $\leftarrow$  True
7:       break
8:     end if
9:   end for
10: end if
11: end for
12: return Result

```

between pages, frequent redundant invocations of identical APIs may occur. To address this, during the storage of access records, we implement a hash-based deduplication method that validates uniqueness based on API request parameters and paths, thereby filtering redundant requests.

Finally, *WAScope* integrates an interactive visualization dashboard that supports the real-time display of detection results and enables the export of leakage detection reports.

4. Implementation

WAScope is developed in Python [33] and consists of four components:

- **Traffic Intercept Proxy Component:** Based on the MITMproxy [34] (man-in-the-middle proxy) framework, it intercepts HTTP(S) traffic and integrates the BeautifulSoup4 [35] library to parse the structure of HTML/XML messages, with approximately approximately 250 SLOC (Source Lines of Code).
- **Privacy Analysis Component:** By constructing a privacy dictionary, it matches and identifies privacy data in the parameters, paths, and response contents of web application-specific APIs, with approximately approximately 260 SLOC.
- **API Confusion Component:** As a core module, this component performs obfuscation operations on the paths and parameter values of web application-specific APIs; it automates request replay using the *requests* network library, with approximately approximately 400 SLOC.
- **Leak Detection Component:** It first performs content verification between original and confused responses via differential hash comparison, and then detects privacy leakage based on the privacy dictionary, with approximately approximately 100 SLOC.

In addition to the four components, *WAScope* has the following modules:

- **Data Module:** A structured visited-record storage system is implemented using the MySQL [36] database. The storage service is developed in Python and consists of approximately approximately 200 SLOC.
- **Visualization module:** Implemented based on the Flask [37]+Vue.js [38] + Element Plus [39] technology stack, it comprises of a RESTful API interface (approximately 80 SLOC) and a front-end dashboard (approximately 760 SLOC), providing interactive result presentation and report generation. This module supports real-time viewing of detection results and exporting of detailed reports.

The total codebase of *WAScope* (excluding third-party dependencies) comprises approximately 2000 SLOC. Each component is implemented using standardized object-oriented programming practices, conforming to modern software engineering quality standards.

We have publicly released *WAScope*,¹ including the privacy dictionary, on GitHub to support future research efforts.

5. Evaluation

We performed experiments to evaluate the capability of *WAScope* in detecting privacy data leakage, and our evaluation targets the following research questions:

- **RQ1:** How can we evaluate *WAScope* detection capability and accuracy in identifying privacy leakages in web application-specific APIs in the real world? (Section 5.2)
- **RQ2:** How practical are localized synonyms in privacy dictionaries for privacy analysis? (Section 5.3)
- **RQ3:** How effective is *WAScope* in identifying privacy-aware APIs and detecting data leakage, compared to state-of-the-art tools? (Section 5.4)
- **RQ4:** How valuable is API confusion in unauthorized access detection? (Section 5.5)
- **RQ5:** What effective defense mechanisms should be constructed to address the privacy data leakage issues revealed by *WAScope*? (Section 5.6)

¹ <https://github.com/jcifax/WAScope.git>

5.1. Experiment setup

(1) Comparison Tools. We compare *WAScope* with several popular web API vulnerability detection and API testing tools. Notably, these tools are designed for different purposes. Vulnerability detection tools primarily aim to discover general security vulnerabilities in web applications. In contrast, API testing tools are designed to generate comprehensive API test cases to uncover functional bugs (e.g., server 500 errors). However, these tools are not explicitly designed to detect privacy data leakage in web application-specific APIs.

We focus on different capabilities of *WAScope* and compare them with the corresponding strengths of each selected tool to address our evaluation goals. For example, can *WAScope* identify more specific APIs that request privacy data?

Based on these considerations, we selected two vulnerability detection tools and two API testing tools for comparison:

- **VOAPI²**[15] is a tool designed for vulnerability detection in RESTful APIs. It identifies and reports security vulnerabilities using standardized keywords and rule-based matching techniques.
- **Sheriff**[16] employs static analysis and dynamic taint tracking to detect client-side request hijacking vulnerabilities. It is particularly effective in identifying privacy data leakage and access control flaws in modern web applications.
- **OWASP ZAP**[13] is an active scanning and testing tool for APIs. It can identify potential data leakage risks with a certain degree of effectiveness, especially when used for dynamic security assessments.
- **Burp Suite**[14] is widely used to intercept and analyze API requests and responses for comprehensive API testing. When configured with appropriate plugins, it can also perform deep exploration of potential security vulnerabilities.

(2) Experimental Dataset. We construct a multi-jurisdictional dataset consisting of the following:

- **China Samples (n=50):** Chinese government and education web applications from Beijing, Hubei, and Jiangxi provinces.
- **Outside China samples (n=50):** Government and education web applications from Tranco list (ID:4KYXX, 2023-09-30) [40] downloaded on September 30, 2023.

This cross-regional sampling strategy offers a broad perspective, enabling a comparative analysis of data security practices across jurisdictions. All selected web applications allowed open registration. We registered user accounts for each application and evaluated them using *WAScope*.

5.2. Detection capabilities (RQ 1)

Experimental Findings. Through the use of hash-based deduplication, *WAScope* intercepted a total of 320,581 specific API requests during a systematic evaluation of 100 web applications, flagging 15,593 privacy-aware APIs. Following the obfuscation operations, 546,183 replay requests were issued, and 3029 APIs were identified as potentially at risk for privacy leakage.

Detection analysis of these requests and statistical evaluation of the response data revealed widespread exposure of privacy data items across 76 web applications, including personally identifiable information such as real names, gender, and national identification numbers.

We categorized the detected privacy data items into four groups: Chinese government services, Chinese education services, non-Chinese government services, and non-Chinese education services. We also recorded the number of web applications leaking each corresponding privacy field, as detailed in Table 2.

Validation Methodology. We manually validated the flagged APIs using the reports described in Section 3.5 to ensure detection accuracy. Our validation procedure included the following steps:

1. Role-switching tests in collaboration with senior developers;
2. Live session reproduction using authenticated access via Edge DevTools [41] and Apifox [42], combined with parameter confusion techniques;
3. Differential response analysis to confirm privacy leakage detection capabilities.

Hash Parameter Handling. During testing, we observed that certain web applications employed hashed values for critical request parameters, which rendered API confusion techniques less effective or ineffective.

Testing Methodology:

1. Authenticated as *Alice* and capture her authorization-related hash parameter.
2. Authenticated as *Bob* and substitute his hash parameter with the value captured from *Alice*.

This parameter substitution technique allowed us to assess potential authorization flaws through the following observations:

1. Whether the system correctly validated user session identifiers;
2. Whether access to restricted resources remained properly isolated between accounts;
3. Whether any unauthorized cross-account data exposure occurred.

This method was designed to uncover authorization bypass vulnerabilities that could result in improper access to user-specific data across accounts.

Manual Verification Results. We manually verified 2757 specific APIs flagged by *WAScope*, and confirmed that 3029 APIs were indeed at risk of privacy leakage, including additional variants discovered during testing.

Additionally, we identified 272 APIs exhibiting unauthorized access behavior; however, their privacy fields were masked or anonymized, resulting in no actual privacy leakage.

Taking both verified leakages and masked responses into account, the estimated false positive rate of *WAScope* is approximately 9%.

Detection Insights. During the of high-risk APIs, we made several additional observations:

- **Request Methods:** In web applications, GET requests are the most prevalent among confused API requests that return privacy data, accounting for 72.67% of all cases. Among over 10,000 GET request instances across 100 web applications, we observed that some GET requests do not require any parameters and can retrieve complete user privacy data in a single call. These cases typically occur during interactions with business-related user modules. For POST requests, the payload format was divided into JSON payloads (47.89%) and form data submissions (52.11%). Additionally, we identified a total of 14,812 asynchronous requests, comprising approximately 95% of all requests, while synchronous requests totaled 781 (5%). Detailed statistics are presented in Table 3.
- **Content Types:** In evaluating the content types of response data, we found that most privacy data is returned in JSON format. However, we also discovered a novel method of privacy leakage: some web applications embed user privacy data within dynamically generated images (e.g., student IDs, personal loan details, or financial information). Although this approach is intended to prevent client-side DOM-based tampering, a simple OCR (Optical Character Recognition) operation on the response image can still extract substantial amounts of user privacy data. Detailed results are shown in Table 4.

Table 2
Privacy data fields with leakage risks detected by WAScope.

Privacy data fields		Web application categorization			
		China		Outside-China	
		Government	Educational	Government	Educational
Personal identity information	Real name	20	23	17	16
	Gender	18	20	17	16
	Nationality	16	14	13	11
	Identity card number	20	23	17	16
	Driver's license number	0	0	6	0
	Social security number	6	0	14	0
	Housing provident fund <i>Only China</i>	1	0	0	0
	Birthday	20	23	17	16
	Religious beliefs	8	0	6	2
	Marital status	16	0	13	0
	Contact address	20	23	17	16
	Educational	17	23	12	16
	Employment	20	14	17	9
	Photographic	10	20	6	10
Social communication	Email	20	23	17	16
	Telephone	20	23	15	12
	Mobile	20	23	15	12
	QQ	0	0	0	0
	WeChat	0	0	0	0
	twitter	0	0	0	0
	Facebook	0	0	0	0
	Instagram	0	0	0	2
Finance	Bank accounts	10	10	12	0
	Alipay	0	0	0	0
	Wechat pay	0	0	0	0
	Insurance	0	0	5	1
	Paypal	0	0	5	0
Documents	Authentication	6	2	0	0
	Certification	1	0	0	0
	Other	2	5	3	8

The number in the cell is the statistical number of web applications that leaked this privacy.

Table 3
Summary of original request APIs, Confused APIs and Leakage APIs situation.

	Request methods									Total request		
	GET			POST (form data)			POST (payload)					
	Original	Confused	Leakage	Original	Confused	Leakage	Original	Confused	Leakage	Original	Confused	Leakage
Async	10,673	384,228	2028	2157	69,024	475	1982	63,424	238	14,812	516,676	2740
Sync	658	26,320	17	69	1794	0	54	1393	0	781	29,507	17
Total	11,331	410,548	2045	2226	70,818	475	2036	64,817	238	15,593	546,183	2757

Original: Number of original APIs, Confused: Number of confused APIs, Leakage: Number of manual confirms leakage APIs.

Table 4
Summary of original APIs response ContentType.

	Request methods									Total response Content-Type		
	GET			POST-form data			POST-payload					
	HTML	JSON	Image	HTML	JSON	Image	HTML	JSON	Image	HTML	JSON	Image
Async	107	10,557	9	462	1692	3	226	1756	0	795	14,005	12
Sync	658	0	0	23	46	0	15	39	0	696	85	0
Total	765	10,557	9	485	1738	3	241	1795	0	1491	14,090	12

Table 5
Summary of Chinese localized synonyms in China web APPs.

Words	Parameters (POST&GET)	Path	Response fields
Synonyms words	3	4	113
Total words	65	97	427
Percent	4.61%	4.12%	26.46%

5.3. Localized synonyms (RQ 2)

Table 5 presents the proportion of Chinese-localized synonyms observed in 50 Chinese web applications after deduplicating terms found in parameters, paths, and response data fields.

To empirically validate the role of localized synonyms in privacy analysis, we removed all Chinese-localized synonyms from the privacy dictionary.

We then replicated the penetration tests across the 50 Chinese web applications. The results, summarized in Table 6, revealed three key impacts: (1) a measurable decline in the detection of privacy-aware

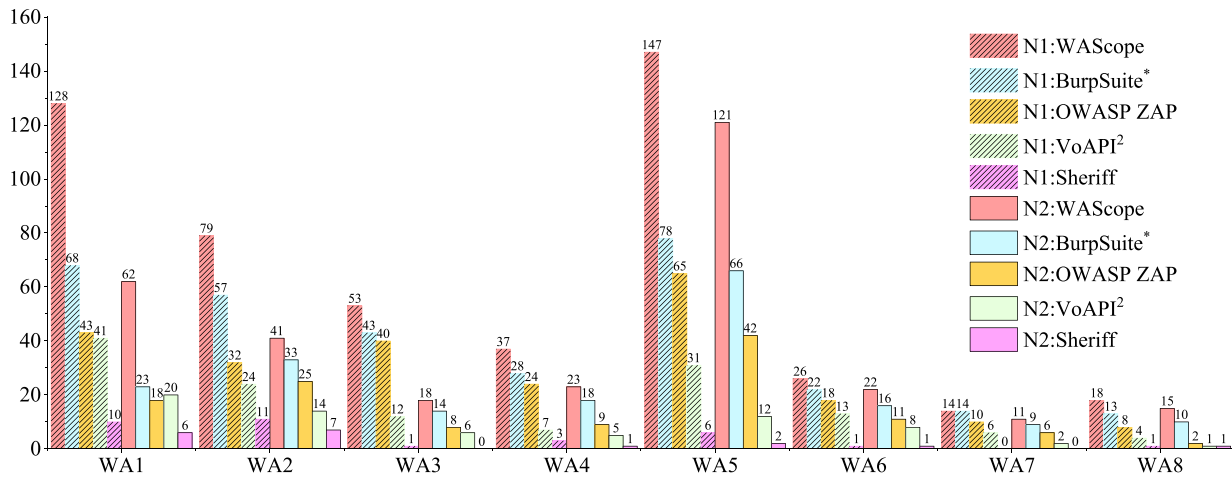


Fig. 5. Comparison of specific-APIs test.

Legend: N1 = Number of privacy-aware APIs; N2 = Number of APIs with privacy leakage.
WA1 = Web application 1, WA2–WA8 follow similarly. Burp Suite* with the APIKit plugin.

Table 6

Capability of localized synonyms.

Type	N1	N2	N3
Dict ⁺	11,155	2071	43
Dict ⁻	7216	1329	40

Dict⁺ = Privacy dictionary with Chinese synonyms,

Dict⁻ = Privacy Dictionary removed Chinese synonyms,

N1 = Number of privacy-aware APIs,

N2 = Number of APIs with privacy leakage,

N3 = Number of leakage web APPs.

APIs, (2) a reduction in the accuracy of privacy leakage identification, and (3) the omission of several at-risk web applications from detection.

5.4. Comparative experiment (RQ 3)

To evaluate *WAScope* against state-of-the-art tools, we selected eight web applications: two government and two education applications from China, and two government and two education applications from outside China. These were chosen based on having the lowest number of privacy-leaking APIs in our dataset, thus serving as a conservative benchmark for comparison.

This evaluation aimed to determine whether other tools could identify more privacy-leaking APIs than those detected by *WAScope*.

The experimental results demonstrate the superior capability of *WAScope* in identifying privacy-data-requesting APIs. Across the eight applications, it detected 502 privacy-aware APIs, of which 313 were confirmed to contain privacy leakage vulnerabilities, as illustrated in Fig. 5.

VoAPI² primarily focuses on RESTful APIs and relies on a limited set of keywords, which are less comprehensive than the privacy and sensitive term dictionary used by *WAScope*. Consequently, it performs poorly in identifying privacy-sensitive APIs and detecting leakage risks compared to *WAScope*.

Sheriff is designed to detect client-side requests initiated through system-level web APIs. It exhibits limited capability in identifying web application-specific APIs that request privacy data and in detecting associated leakage risks.

OWASP ZAP features active scanning and API discovery capabilities. However, it is restricted to detecting standard privacy fields and fails to identify complex or application-specific data formats containing

sensitive information. Its detection performance is therefore inferior to that of *WAScope*.

When the APIKit [43] plugin is configured for **Burp Suite**, the tool can passively capture APIs that contain privacy data. Manual inspection may then uncover data leakage instances. However, in both automated detection capabilities and the comprehensiveness of privacy leakage discovery, Burp Suite underperforms compared to *WAScope*.

To better understand the positioning of *WAScope*, we compare it with both industrial and academic tools in Table 7. Burp Suite and OWASP ZAP are widely adopted for general web security assessments, yet they lack privacy-specific capabilities such as localized dictionary support and differential analysis. VoAPI² focuses on RESTful APIs using rule-based keyword matching, but lacks sufficient semantic coverage for identifying privacy-sensitive APIs. Sheriff employs static and taint analysis to detect client-side request hijacking but is not suited for detecting API-level privacy leakage within server-side web applications.

In contrast, *WAScope* combines dynamic confusion-based request generation with a configurable privacy dictionary and a differential analysis engine, enabling precise detection of unauthorized data exposure in real-world web applications.

5.5. Ablation experiment (RQ 4)

In the ablation experiments, we conducted a detailed breakdown of the obfuscation operations applied to specific APIs to evaluate the individual effectiveness of each strategy.

We used the same eight web applications from the comparative experiment for this penetration testing. The impact of the enumeration, incremental, and random deletion operations on unauthorized access detection was systematically analyzed.

As shown in Table 8, no risky APIs were detected when API confusion was not applied. Upon introducing the enumeration operation, we were able to detect not only APIs at risk of unauthorized access, but also those that exposed resources beyond the current user's permissions. Subsequently, incorporating the incremental operation led to a notable increase in the detection of unauthorized API access risks. Finally, applying the random deletion operation further contributed to identifying unauthorized APIs, particularly those adhering to RESTful specifications.

5.6. Defensive measures (RQ 5)

Privacy data leakage is a prominent security issue in web applications. Therefore, how can we defend against privacy data leakage problems caused by unauthorized access while ensuring system performance? We propose five primary countermeasures as follows:

Table 7
Feature-level comparison of WAScope with SOTA tools.

Feature	Burp suite	OWASP ZAP	VoAPI ²	Sheriff	WAScope
Privacy dictionary support	○	○	○	○	●
Confusion-based API mutation	○	○	○	○	●
Differential response analysis	○	○	○	○	●
Automated replay of API requests	◐	◐	○	○	●
Dynamic testing on live web apps	●	●	○	○	●
Localized semantics support	○	○	○	○	●
Privacy leakage detection focus	○	○	●	●	●

○ Not support, ◐ Partially support, ● Support.

Table 8
Obfuscation operations impact on UAVs.

Operations	N1	N2
None	0	0
Enum.	1356	53
Incr.	3465	0
R.D.	392	24
Enum. + Incr	4821	53
Enum. + R.D.	1748	77
Incr. + R.D.	3857	24
Enum. + Incr. + R.D.	5213	77

N1 = Number of APIs with unauthorized access risk.

N2 = Number of APIs not within the current user's permissions.

1. **Remove unnecessary information:** By limiting the data included in APIs to the minimum scope required for corresponding business needs, we can effectively and confidently prevent information leakage.
2. **Limit API access frequency:** By strictly limiting the maximum number of API calls within a specific time frame, we effectively prevent large-scale data extraction.
3. **Enhance API call authorization:** Every API call should trigger verification of the caller's credentials to ensure that the requester possesses the necessary permissions for the operation.
4. **Enhancing token usage and management:** Implementing one-time tokens or setting expiration policies ensures tokens become invalid within a short period, significantly reducing the risk of token reuse even if intercepted.
5. **Path and parameter obfuscation:** Encrypting or obfuscating API paths and parameters increases the difficulty for attackers to exploit these details in attacks.

Finally, we demonstrate the effectiveness of these defense measures through four case studies of fixed privacy data leakage vulnerabilities.

Case 1: A government web application within China. After logging into the system, WAScope identified a web application-specific API path containing the sensitive keyword “account”, such as `/subAccount/Mx45`. The response data included multiple items of personally identifiable information (PII) related to sub-accounts, including real names, account names, and passwords. Through path confusion, identity information for different users' sub-accounts was successfully retrieved, without any request being blocked by the server.

After discovering these issues, we proactively reported them to CNVD and received CNVD IDs CNVD-2023-84884 and CNVD-2024-29728. We recommended the five defense measures described earlier. After regression testing confirmed that the vulnerability had been resolved, the web application implemented the following two measures:

1. **Remove unnecessary information:** The MD5 password field was removed from the API response.
2. **Enhance API call authorization:** Accessing other users' sub-account information is no longer possible.

Case 2: An educational project web application within China. After logging into the system, we identified an API path containing the sensitive keyword “UserQuery”. This request required no parameters, yet its

response disclosed nearly 20,000 user records in plain text, including users' real names, national ID numbers, and organization identifiers. Leveraging this information, we were able to retrieve the complete data of other users during business processes by simply substituting different user IDs.

After discovering these issues, we proactively reported them to CNVD and received the CNVD ID CNVD-2023-73406. We recommended the five previously discussed defense measures. Following regression testing confirming successful remediation, the web application implemented the following three countermeasures:

1. **Remove unnecessary information:** Expert-related data was removed from the approval progress interface, preventing access to peer review information.
2. **Enhance API call authorization:** Access to other users' project and personal information is now restricted.
3. **Enhancing token usage and management:** User tokens are now bound to specific devices and expire within a predefined validity period.

Case 3: A financial service web application widely used by government and educational institutions within China. After logging into the system, we identified user IDs exposed within the API request parameters. The corresponding API response returned sensitive personal data in plaintext, including real names, student or employee numbers, organization identifiers, bank card numbers, and phone numbers. By enumerating user IDs, we were able to extract sensitive information of other users.

After discovering these issues, we proactively reported them to the China National Vulnerability Database of Information Security and received CNVD ID CNVD-2024-13965. We proposed the previously outlined five defensive measures. Upon regression testing and confirmation of the vulnerability's resolution, the web application adopted one of these measures, as detailed below:

1. **Remove unnecessary information:** Student and employee identifiers were removed from business data retrieval APIs, and sensitive fields such as bank card numbers and phone numbers were properly masked.

Case 4: An overseas government web application listed on the Tranco list. The user ID appeared in both the parameters and the path of specific API requests, while the corresponding API responses disclosed personally identifiable information, including users' personal and residential addresses. By enumerating user IDs and mutating the API path, we were able to extract privacy data of different users. Upon discovering these issues, we voluntarily reported them to the application's administrators. Following our recommended defensive measures and subsequent patch verification, the application adopted one of them, described below:

1. **Enhance API call authorization:** Access to other users' project and personal information is now restricted.

These four case studies demonstrate that the five proposed defensive measures can effectively mitigate privacy data leakage risks in web application-specific APIs.

6. Discussion

In this section, we critically examine the limitations of the current *WAScope* prototype, as well as opportunities for generalization, scalability, and real-world deployment. These insights are intended to both acknowledge the boundaries of our current implementation and outline practical strategies for extending the system.

6.1. Generalizability of the privacy dictionary

Although the privacy dictionary is designed with a modular and extensible architecture, its current implementation primarily supports Chinese and English web applications. Localization for other regions—such as identifiers used in India (e.g., “*AadhaarNum*”) or Japan (e.g., “*MyNumber*”) is not included by default. These extensions can be easily integrated into the JSON structure described in Section X, but require manual augmentation by users. Future work may focus on building regional dictionary modules and automatic schema extraction from open government datasets.

6.2. Limitations of the API confusion strategy

Our current obfuscation-based confusion strategy—comprising enumeration, incremental substitution, and random parameter deletion—is effective in exposing loosely protected endpoints. However, it remains ineffective against scenarios where parameters are encrypted, hashed, or otherwise obfuscated by the application logic.

This limitation is particularly relevant in web applications that adopt client-side encryption, where sensitive fields (e.g., identity number, phone number) are encrypted using symmetric algorithms (e.g., AES) before being transmitted. Since such encryption typically occurs on the client side, and the keys or transformation logic are embedded in JavaScript, there is potential for automated reverse engineering or browser-based instrumentation to intercept or recover plaintext values prior to encryption.

Although *WAScope* does not currently support this functionality, we are actively exploring techniques for automated decryption of user-controlled parameters. Specifically, we aim to analyze encryption routines within the browser context and extract pre-encryption values using headless browser instrumentation (e.g., Puppeteer-based hooking). This enhancement will enable *WAScope* to detect leaks that are currently masked by client-side encryption.

Additionally, parameters obfuscated via one-way hash functions pose a different challenge. In our experiments, we manually registered two test accounts, captured their respective hashed identifiers, and performed controlled hash parameter replacement tests to detect shared-access vulnerabilities. This manual method proved effective in revealing flaws hidden behind hashed identifiers. Moving forward, we plan to integrate this multi-account hash testing mechanism into *WAScope*, allowing for automated testing of hashed parameters across multiple user contexts. This will further enhance the tool’s coverage in applications using deterministic but obscured identifiers.

6.3. False negative analysis

While *WAScope* achieves a low false positive rate (9%), the tool may still suffer from false negatives—cases where privacy data leakage exists but is not detected. This section provides a preliminary analysis of such undetected vulnerabilities and outlines the challenges involved in recall estimation.

Several factors can lead to false negatives in real-world deployments:

- **Dynamic token validation:** Short-lived or one-time-use tokens may expire before the confusion tests are executed, leading to misleadingly consistent responses.
- **Rate limiting or anomaly detection:** Target servers may throttle or block requests if they detect automated behavior, thus masking differences in access control behavior.
- **Server-side caching:** Caching mechanisms can cause identical responses to be returned across different user contexts, hiding leakage vulnerabilities.
- **POST body obfuscation:** Privacy-sensitive identifiers embedded in the request body—especially when encoded or encrypted—are harder to locate and mutate effectively.
- **Non-JSON responses with embedded sensitive data:** In some cases, server responses are not structured JSON but rather image formats (e.g., PNG, JPG) containing visually rendered sensitive data such as ID cards or user information. Since *WAScope* currently focuses on textual content diffing, such binary content is not parsed, resulting in missed detections.

To gain insight into potential false negatives, we manually reviewed 100 APIs that were not flagged by *WAScope*. Among them, we found several endpoints where access was controlled by expiring session tokens or where sensitive data was returned in image-based formats. These observations confirm that *WAScope* detection capability may be affected in such conditions.

Although establishing a complete ground truth for all vulnerable APIs is inherently difficult, these findings motivate future enhancements. Planned improvements include timing-aware token synchronization and support for binary response analysis (e.g., content-type detection and image-based OCR) to reduce false negatives.

6.4. Performance overhead

To evaluate the performance overhead of *WAScope*, we measured its runtime cost during the large-scale evaluation involving 546,183 replayed API requests. All experiments were conducted on a workstation with an Intel Core i7-11700 CPU (8 cores, 2.5 GHz), 32 GB RAM, and Ubuntu 20.04. The tool was executed in single-threaded mode for consistency.

On average, each API confusion took 0.82 s to complete across the three strategies. The total runtime was approximately 185 h, yielding an average throughput of 0.82 requests per second. We found that network latency, rather than local computation, dominated the processing time, and resource usage remained stable.

All tests were conducted during off-peak hours and in coordination with system administrators, ensuring no disruption to production services. Moreover, since *WAScope* is intended for pre-deployment testing, performance optimization was not the primary design goal.

6.5. Defense mechanism validation

To empirically validate the defense strategies proposed in Section 5.6, we conducted follow-up testing for the web applications used in Case Studies 1–4.

After reporting the confirmed vulnerabilities to the respective administrators, we waited one month and then performed regression testing using the same *WAScope* configuration, including all three confusion strategies. The results showed that the leakage risks previously identified had been resolved.

This indicates that the administrators likely adopted our recommended defenses, such as enforcing token expiration, introducing request signature validation, and obfuscating sensitive parameters.

Although we did not have direct access to their deployment logs or patch records, the elimination of leakage signals under the same testing conditions provides strong empirical support for the efficacy of our defense recommendations.

6.6. Protocol coverage limitation

Currently, *WAScope* supports API traffic over HTTP/HTTPS, which remain dominant in contemporary web application development.

However, emerging API technologies such as gRPC and GraphQL are increasingly adopted in microservices and large-scale platforms. At this stage, *WAScope* does not support these protocols due to their different request structures (e.g., protobuf encoding in gRPC, nested query structures in GraphQL).

In future work, we plan to extend *WAScope* to incorporate protocol-aware parsing and confusion testing for gRPC and GraphQL. This will involve augmenting the request generation engine, adjusting parameter mutation logic, and adapting the differential analysis mechanism accordingly.

6.7. Real-world deployment challenges

In real-world scenarios, automated testing tools often encounter barriers such as CAPTCHA prompts, bot detection mechanisms, or session throttling. These can hinder large-scale replay or provoke blocking behavior from security middleware.

To address this, *WAScope* adopts several mitigation techniques:

- **Randomized request delays** are introduced to emulate human behavior and reduce the likelihood of triggering rate-based defenses.
- **Parameter mutation is constrained** to prevent combinatorial explosion and to avoid behavior characteristic of automated attacks.
- **Selective data capture** is applied: *WAScope* collects only a minimal subset of API responses necessary to determine whether sensitive data was leaked. It does not perform large-scale scraping or retention of private information.

These strategies proved effective during our evaluations. In future versions, we aim to integrate **headless browser support** to improve compatibility with client-side anti-bot logic.

7. Related work

The issue of privacy data leakage has a long history and has been the subject of numerous research efforts in the past. Below, we summarize relevant prior work and compare it with our approach.

In the field of unauthorized access vulnerability detection, Marc Rennhard [44] proposed a practical solution for automatically detecting access control vulnerabilities in HTTP GET requests within web applications. In contrast, *WAScope* not only analyzes GET requests but also incorporates POST (form data and payload), PUT, and DELETE requests. Our approach provides broader test coverage for web applications and a wider range of HTTP request types, significantly exceeding the scope of their proposed solution.

In research on vulnerabilities in authentication and authorization mechanisms, studies have focused on CSRF [12,45–47] and client-side CSRF [48,49]. Given the severity of these threats, extensive efforts have been devoted to developing countermeasures [50–53]. Other studies have investigated replay attack vulnerabilities [54,55]. Distinct from these efforts, *WAScope* leverages valid session credentials post-authentication and generates forged API requests by manipulating parameters and paths (i.e., API confusion). These replayed requests enable broader extraction of sensitive data while evading detection, as they originate from legitimate user sessions recognized by the server.

In the area of Web API security, Khodayari [16] investigated hijacking vulnerabilities in system APIs used by web clients, focusing on APIs defined by W3C and WHATWG specifications. In contrast, our work targets application-specific APIs embedded in real-world web applications. Wenlong Du [15] studied vulnerabilities in RESTful APIs,

relying on common keywords for API tracking and detection. *WAScope* adopts a broader scope that includes both RESTful and non-RESTful APIs. Furthermore, our privacy dictionary is significantly more comprehensive, enabling superior detection performance in identifying privacy data leakage compared to their approach.

In the domain of API testing, several widely-used tools exist, such as Burp Suite [14] and Apifox [42], which are commonly used in web security assessments. Among these, Burp Suite is the most commonly adopted. However, its limited privacy data identification capabilities—primarily due to a less comprehensive dictionary—result in weaker detection performance for privacy-aware APIs. In both privacy API discovery and leakage detection, *WAScope* significantly outperforms Burp Suite.

8. Conclusion

We conclude our work on *WAScope* with the following key findings, drawn from each major section of the study:

- **Privacy-Aware API Identification:** We constructed a multi-lingual privacy dictionary and intercepted HTTP(S) traffic to detect API requests that handle sensitive user data. This formed the foundation for automated privacy leakage detection in web applications.
- **Obfuscation via API Confusion:** A three-phase confusion strategy—enumeration, incremental mutation, and random deletion—was designed to generate diverse API variants. These obfuscations significantly improved the ability to trigger unauthorized access paths during testing.
- **Leakage Detection Mechanism:** *WAScope* used response differential analysis and hash comparison to detect leakage. Manual validation confirmed 2757 vulnerable APIs with a low false positive rate (9%).
- **Comparative and Ablation Study:** Our evaluation demonstrated that *WAScope* outperforms four state-of-the-art tools in discovering privacy-exposing APIs. Ablation studies confirmed the contribution of each confusion phase.
- **Localized Synonym Effectiveness:** Removing localized synonyms from the dictionary resulted in significant degradation in detection capability across Chinese web applications, emphasizing their necessity.
- **Real-World Impact:** *WAScope* identified 15,593 privacy-aware APIs across 100 real-world web applications. These findings led to the discovery of systemic vulnerabilities, with over 10 confirmed disclosures.

Ethical Compliance: All tested web applications were anonymized, and penetration testing was conducted with prior authorization from system administrators. Testing was scheduled during off-peak hours to avoid disruption of normal services. All identified vulnerabilities were responsibly disclosed to the affected parties, and any collected data were securely deleted following validation.

In addition, our findings were submitted to the China National Vulnerability Database (CNVD) through its coordinated disclosure process. As of this submission, ten vulnerabilities have been confirmed and assigned official CNVD identifiers: CNVD-2023-73406, CNVD-2023-84884, CNVD-2024-13965, CNVD-2024-23734, CNVD-2024-29728, CNVD-2024-46167, CNVD-2024-44467, CNVD-2025-01252, CNVD-2025-01253, and CNVD-2025-04769. Additional disclosures are currently under review.

CRedit authorship contribution statement

Yu Nie: Writing – original draft, Visualization, Validation, Software, Funding acquisition. **Jianming Fu:** Supervision, Methodology. **Xinghang Lv:** Resources, Investigation. **Chao Li:** Validation. **Shixiong Yang:** Validation. **Guojun Peng:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (62272351), Jiangxi Province 03 Special Project and 5G Project (No. 20232ABC03A29). We would like to thank the anonymous reviewers for their insightful comments and suggestions to improve our paper.

References

- [1] Forbes Media LLC, Top website statistics for 2024, 2024, <https://www.forbes.com/advisor/business/software/website-statistics/>, Last accessed on 2025-03-07.
- [2] R. Flores, C. Perine, L. Remorin, R. Reyes, Examining security risks in logistics APIs used by online shopping platforms, 2022, https://documents.trendmicro.com/assets/white_papers/wp-examining-the-security-risks-in-logistics-apis-used-by-online-shopping-platforms.pdf, Last accessed on 2025-03-07.
- [3] Southern Metropolis Daily (SMD), More than 30,000 sensitive information of teachers and students leaked in a university in nanchang, 2023, <https://new.qq.com/rain/a/20230817A016U200>, Last accessed on 2025-03-07.
- [4] China Central Television (CCTV), A government information system technology service company in Shanghai city has been administratively punished for selling citizens' personal information disclosed overseas, 2023, <https://news.cctv.cn/2023/09/15/ARTI9iO75vnmfCJ7jqpxkOIP230915.shtml>, Last accessed on 2025-03-07.
- [5] The Paper, Failure to fulfill data security protection obligations, these companies are fined, 2023, https://www.thepaper.cn/newsDetail_forward_24519257, Last accessed on 2025-03-07.
- [6] OWASP, Top 10 web application security risks, 2021, <https://owasp.org/www-project-top-ten/>, Last accessed on 2025-03-07.
- [7] S. Peltonen, L. Mezzalana, D. Taibi, Motivations, benefits, and issues for adopting micro-frontends: A multivocal literature review, *Inf. Softw. Technol.* 136 (2021) 106571, <http://dx.doi.org/10.1016/j.infsof.2021.106571>, URL <https://www.sciencedirect.com/science/article/pii/S0950584921000549>.
- [8] W3C, W3C standards and drafts, 1994, <https://www.w3.org/TR/>, Last accessed on 2025-03-07.
- [9] WHATWG, WHATWG specifications, 2004, <https://spec.whatwg.org>, Last accessed on 2025-03-07.
- [10] Mozilla, XMLHttpRequest API, 2024, <https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest>, Last accessed on 2025-03-07.
- [11] WHATWG, Fetch API, 2024, <https://fetch.spec.whatwg.org/>, Last accessed on 2025-03-07.
- [12] G. Pellegrino, M. Johns, S. Koch, M. Backes, C. Rossow, Deemon: Detecting CSRF with dynamic analysis and property graphs, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 1757–1771, <http://dx.doi.org/10.1145/3133956.3133959>, URL <https://doi.org/10.1145/3133956.3133959>.
- [13] OWASP, OWASP ZAP, 2012, <https://www.zaproxy.org/>, Last accessed on 2025-03-07.
- [14] PortSwigger Ltd, Burpsuite, 2018, <https://portswigger.net/burp>, Last accessed on 2025-03-07.
- [15] W. Du, J. Li, Y. Wang, L. Chen, R. Zhao, J. Zhu, Z. Han, Y. Wang, Z. Xue, Vulnerability-oriented Testing for RESTful APIs.
- [16] S. Khodayari, T. Barber, G. Pellegrino, The great request robbery: An empirical study of client-side request hijacking vulnerabilities on the web, in: Proceedings of 45th IEEE Symposium on Security and Privacy, 2024.
- [17] mozilla, Notifications, 2024, <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/notifications>, Last accessed on 2025-03-07.
- [18] Mozilla, Websockets, 2023, <https://developer.mozilla.org/en-US/docs/Glossary/WebSockets>, Last accessed on 2025-03-07.
- [19] Mozilla, sendBeacon, 2024, <https://developer.mozilla.org/en-US/docs/Web/API/Navigator/sendBeacon>, Last accessed on 2025-03-07.
- [20] J.A. Díaz-Rojas, J.O. Ocharán-Hernández, J.C. Pérez-Arriaga, X. Limón, Web API security vulnerabilities and mitigation mechanisms: A systematic mapping study, in: 2021 9th International Conference in Software Engineering Research and Innovation, CONISOFT, 2021, pp. 207–218, <http://dx.doi.org/10.1109/CONISOFT52520.2021.00036>.
- [21] I. Hydar, A.B.M. Sultan, H. Zulzalil, N. Admodisastro, Current state of research on cross-site scripting (XSS) — a systematic literature review, *Inf. Softw. Technol.* 58 (2015) 170–186, <http://dx.doi.org/10.1016/j.infsof.2014.07.010>, URL <https://www.sciencedirect.com/science/article/pii/S0950584914001700>.
- [22] S. Gupta, B.B. Gupta, Cross-site scripting (XSS) attacks and defense mechanisms: Classification and state-of-the-art, *Int. J. Syst. Assur. Eng. Manag.* 8 (2017) 512–530.
- [23] Z. Ye, Y. Guo, A. Ju, F. Wei, R. Zhang, J. Ma, A risk analysis framework for social engineering attack based on user profiling, *J. Organ. End User Comput.* 32 (2020) 37–49.
- [24] D. Irani, M. Balduzzi, D. Balzarotti, E. Kirda, C. Pu, Reverse social engineering attacks in online social networks, in: Detection of Intrusions and Malware, and Vulnerability Assessment: 8th International Conference; DIMVA 2011, Amsterdam, the Netherlands, July 7–8, 2011. Proceedings 8, Springer, 2011, pp. 55–74.
- [25] H. Sanghvi, M. Dahiya, Cyber reconnaissance: An alarm before cyber attack, *Int. J. Comput. Appl.* 63 (2013).
- [26] A. Roy, L. Mejia, P. Helling, A. Olmsted, Automation of cyber-reconnaissance: A Java-based open source tool for information gathering, in: 2017 12th International Conference for Internet Technology and Secured Transactions, ICITST, IEEE, 2017, pp. 424–426.
- [27] T. Koide, D. Chiba, M. Akiyama, To get lost is to learn the way: Automatically collecting multi-step social engineering attacks on the web, in: Proceedings of the 15th ACM Asia Conference on Computer and Communications Security, 2020, pp. 394–408.
- [28] M. Al-Khateeb, M. Al-Mousa, A. Al-Sherideh, D. Almajali, M. Asassfeh, H. Khafajeh, Awareness model for minimizing the effects of social engineering attacks in web applications, *Int. J. Data Netw. Sci.* 7 (2023) 791–800.
- [29] M. Masoud, Y. Jaradat, A.Q. Ahmad, On tackling social engineering web phishing attacks utilizing software defined networks (SDN) approach, in: 2016 2nd International Conference on Open Source Software Computing, OSSCOM, IEEE, 2016, pp. 1–6.
- [30] N.P.C. Standing Committee, Data security law of the People's Republic of China, 2021, http://www.npc.gov.cn/englishnpc/c2759/c23934/202112/t20211209_385109.html, Last accessed on 2025-03-07.
- [31] European Union (EU), General data protection regulation, 2018, <https://gdpr-info.eu/>, Last accessed on 2025-03-07.
- [32] GITEE, Gitee, 2013, <https://gitee.com/>, Last accessed on 2025-03-07.
- [33] Python, Python 3.10, 2021, <https://www.python.org/downloads/release/python-31014/>, Last accessed on 2025-03-07.
- [34] Mitmproxy, Mitmdump, 2021, <https://mitmproxy.org/>, Last accessed on 2025-03-07.
- [35] B. Soup, Beautiful soup documentation, 2021, <https://beautiful-soup-4.readthedocs.io/>, Last accessed on 2025-03-07.
- [36] Oracle, MySQL, 2023, <https://www.mysql.com/>, Last accessed on 2025-03-07.
- [37] Flask, Flask, 2010, <https://flask.palletsprojects.com/>, Last accessed on 2025-03-07.
- [38] E. You, Vuejs, 2023, <https://vuejs.org/>, Last accessed on 2025-03-07.
- [39] Element Plus, Element plus, 2023, <https://element-plus.org/>, Last accessed on 2025-03-07.
- [40] V.L. Pochat, T. Van Goethem, S. Tajalizadehkhoo, M. Korczyński, W. Joosen, Tranco: A research-oriented top sites ranking hardened against manipulation, 2018, arXiv preprint arXiv:1806.01156.
- [41] Microsoft, Microsoft EDGE, 2020, <https://www.microsoft.com/zh-cn/edge/download>, Last accessed on 2025-03-07.
- [42] Guangzhou Ruihu Information Technology Co., Ltd, Apifox, 2021, <https://apifox.com/>, Last accessed on 2025-03-07.
- [43] APIKit, Apikit, 2021, <https://github.com/API-Security/APIKit/>, Last accessed on 2025-03-07.
- [44] M. Rennhard, M. Kushnir, O. Favre, D. Esposito, V. Zahnd, Automating the detection of access control vulnerabilities in web applications, *SN Comput. Sci.* 3 (2022) 376.
- [45] A. Sudhodanar, R. Carbone, L. Compagna, N. Dolgin, A. Armando, U. Morelli, Large-scale analysis & detection of authentication cross-site request forgeries, in: 2017 IEEE European Symposium on Security and Privacy, EuroS & P, 2017, pp. 350–365, <http://dx.doi.org/10.1109/EuroSP.2017.45>.
- [46] S. Calzavara, M. Conti, R. Focardi, A. Rabbitti, G. Tolomei, Mitch: A machine learning approach to the black-box detection of csrf vulnerabilities, in: 2019 IEEE European Symposium on Security and Privacy, EuroS & P, 2019, pp. 528–543, <http://dx.doi.org/10.1109/EuroSP.2019.00045>.
- [47] E. Shernan, H. Carter, D. Tian, P. Traynor, K. Butler, More guidelines than rules: CSRF vulnerabilities from noncompliant OAuth 2.0 implementations, in: M. Almgren, V. Gulisano, F. Maggi (Eds.), Detection of Intrusions and Malware, and Vulnerability Assessment, Springer International Publishing, Cham, 2015, pp. 239–260.
- [48] S. Khodayari, G. Pellegrino, JAW: Studying client-side CSRF with hybrid property graphs and declarative traversals, in: 30th USENIX Security Symposium, USENIX Security 21, 2021, pp. 2525–2542.
- [49] Facebook, Client-side CSRF, 2018, <https://www.facebook.com/notes/facebookbug-bounty/client-side-csrf/2056804174333798/>, Last accessed on 2024-03-07.
- [50] A. Barth, C. Jackson, J.C. Mitchell, Robust Defenses for Cross-Site Request Forgery, CCS '08, Association for Computing Machinery, New York, NY, USA, 2008, pp. 75–88, <http://dx.doi.org/10.1145/1455770.1455782>, URL <https://doi.org/10.1145/1455770.1455782>.

- [51] M. Johns, J. Winter, RequestRodeo: Client side protection against session riding, 2006, URL <https://api.semanticscholar.org/CorpusID:61314725>.
- [52] W. Zeller, E.W. Felten, Cross-site request forgeries: Exploitation and prevention, *N. Y. Times* (2008) 1–13.
- [53] S. Khodayari, G. Pellegrino, The state of the samesite: Studying the usage, effectiveness, and adequacy of samesite cookies, in: *2022 IEEE Symposium on Security and Privacy, SP, IEEE, 2022*, pp. 1590–1607.
- [54] M. Nadeem, A. Arshad, S. Riaz, S.W. Zahra, A.K. Dutta, S. Almotairi, A secure architecture to protect the network from replay attacks during client-to-client data transmission, *Appl. Sci.* 12 (2022) <http://dx.doi.org/10.3390/app12168143>, URL <https://www.mdpi.com/2076-3417/12/16/8143>.
- [55] M.A. Al-Shareeda, S. Manickam, S.A. Laghari, A. Jaisan, Replay-attack detection and prevention mechanism in industry 4.0 landscape for secure SECS/GEM communications, *Sustainability* 14 (2022) <http://dx.doi.org/10.3390/su142315900>, URL <https://www.mdpi.com/2071-1050/14/23/15900>.